Assignment: 1

Name: Piyush Patel
UID:23BCS12673
Branch: BE-CSE
Section/Group: KRG 2-A
Semester: 6th
Date of Submission: 04/02/2026

Subject Name: System Design
Subject Code: 23CSH-314

Ques 1: Explain the role of interfaces and enums in software design with proper examples.

Solution:

An interface defines a contract: it specifies *what a class must do*, not *how it does it*.

- Contains method signatures (and constants)
- No implementation logic (in most languages)
- Classes implement interfaces

Roles of interface

- Abstraction: Interface hides implementation details and expose only behaviour
- Loose Coupling: High-level modules depend on interfaces, not concrete classes

Example: Abstraction and Loose Coupling

```
interface Payment {
    void pay(double amount);   // what must be done
}

// Different implementations, Same contract

class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Credit Card");
    }
}

class UPIPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Payment payment = new CreditCardPayment();  // abstraction
        payment.pay(500);
    }
}

-The client does not know how payment is processed or the internal logic of credit card
/ UPI
-The client only knows pay(double amount) exists
-So, Abstraction achieved via interface
```

An enum (enumeration) is a special data type used to define a fixed set of named constants. It is mainly used when a variable can take only one value from a predefined list

In Java, we do not create objects of an enum using new. Instead, we create references to predefined enum constants, which are automatically created by the JVM.

Example:

```
enum OrderStatus {
    PLACED,
    SHIPPED,
    OUT_FOR_DELIVERY,
    DELIVERED,
    CANCELLED
}
public class Order {
    private OrderStatus status;
    public Order(OrderStatus status) {
        this.status = status;
    }
    public void showStatus() {
        System.out.println("Order Status: " + status);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Order order = new Order(OrderStatus.SHIPPED);
        order.showStatus();
    }
}
```

Roles of enums

- **Type Safety:** Enums provide type safety by restricting a variable to only the values defined in the enum. This prevents invalid or unexpected values at compile time.

  ```
  OrderStatus status = "DELIVERED";   // compile-time error
  OrderStatus status = OrderStatus.DELIVERED;
  ```

  This prevents invalid states, improves reliability, and makes the code easy to understand and maintain.

Ques 2: Discuss how interfaces enable loose coupling with example.

Solution:

Loose Coupling: It means class depends on an abstraction (interface) rather than a concrete implementation. Interfaces enable this by defining what operations are required without specifying how they are performed

Example:

```
interface Payment {
   void pay(double amount);
}
class CreditCardPayment implements Payment {
   public void pay(double amount) {
      System.out.println("Paid " + amount + " using Credit Card");
   }
}
```

```
class UPIPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
}
class OrderService {
    private Payment payment;
    OrderService(Payment payment) {
        this.payment = payment;
    }
    public void checkout(double amount) {
        payment.pay(amount);
    }
}
public class Main {
    public static void main(String[] args) {
        Payment payment = new UPIPayment();  // easily changeable
        OrderService order = new OrderService(payment);
        order.checkout(1000);
    }
}
```

- OrderService is not tightly bound to CreditCardPayment or UPIPayment
- Payment methods can be changed or added without modifying OrderService
- New payment types (e.g., DebitCardPayment) can be introduced easily

Ques 3: Design a HLD for payment processing system, showing where interfaces would be used.

i)Functional Requirements

- User should be able to make payments using – UPI, Credit Card, Debit Card

- System should return payment status - Success, Failed, Pending.
- System should notify user about payment status.
- System should support refund
- System should all log all transactions

ii)Non-Functional Requirements

- Should be able to handle high concurrent transactions
- System uptime > 99.9%
- Secure and encrypted for sensitive data
- Reliable (rollback on failure)
- Maintainability (interface based design, modular components)

iii)API Design

1. **Post**

   POST /api/payments

   ```
   {
     "userId": "U123",
     "amount": 1000,
     "paymentMethod": "UPI"
   }
   ```

   Response

   ```
   {
     "transactionId": "TXN789",
     "status": "SUCCESS"
   }
   ```

2. GET

   GET /api/payments/{transactionId}

iv)Entities

1. User (userId, name, phone, email)

2. Transaction(transactionId, amount, status, payment_Method, timestamp)

3. Notification(notificationId, userId, message, channel)

Payment Interface (for loose coupling and abstraction)

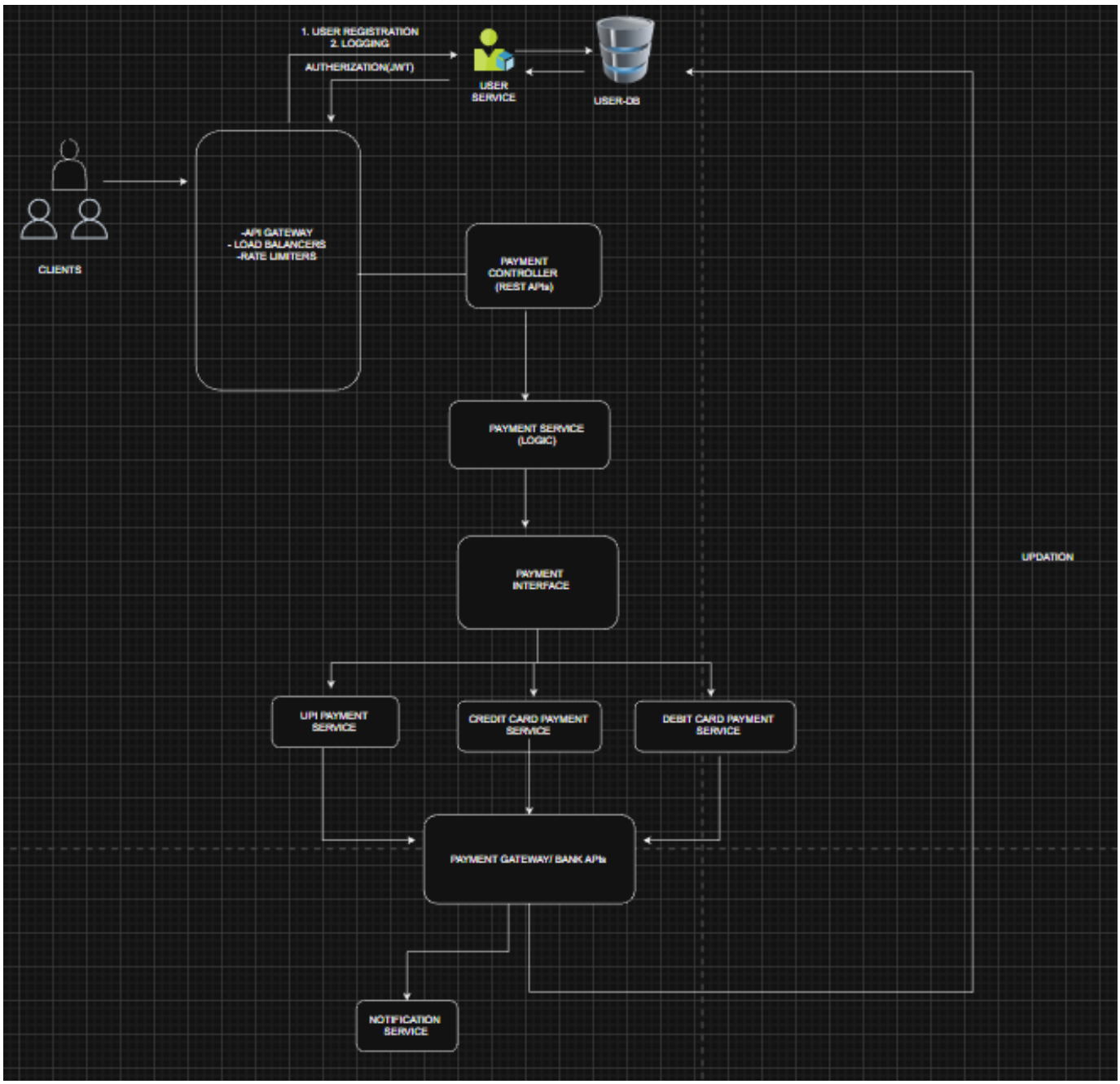
```
public interface Payment {
    PaymentResponse pay(PaymentRequest request);
    PaymentResponse refund(String transactionId);
}

public class UPIPayment implements Payment {
    public PaymentResponse pay(PaymentRequest request) { /* logic */ }
    public PaymentResponse refund(String transactionId) { /* logic */ }
}
public class CreditCardPayment implements Payment {
    public PaymentResponse pay(PaymentRequest request) { /* logic */ }
    public PaymentResponse refund(String transactionId) { /* logic */ }
}
public class DebitCardPayment implements Payment {
    public PaymentResponse pay(PaymentRequest request) { /* logic */ }
    public PaymentResponse refund(String transactionId) { /* logic */ }
}


public class PaymentService {
    private Payment payment;

    public PaymentService(Payment payment) {
        this.payment = payment;
    }

    public PaymentResponse processPayment(PaymentRequest request) {
        return payment.pay(request);
    }
}
```


1. USER REGISTRATION
2. LOGGING
AUTHERIZATION(JWT)
USER SERVICE
USER-DB
CLIENTS
-API GATEWAY
- LOAD BALANCERS
-RATE LIMITERS
PAYMENT CONTROLLER (REST APIs)
PAYMENT SERVICE (LOGIC)
PAYMENT INTERFACE
UPDATION
UPI PAYMENT SERVICE
CREDIT CARD PAYMENT SERVICE
DEBIT CARD PAYMENT SERVICE
PAYMENT GATEWAY/ BANK APIs
NOTIFICATION SERVICE